

WhoDis Software Design Document *

Richard Magnotti

1 Introduction and Overview

This project develops an automated customer support triage system: a tool that powers a business's customer service experience on the user end, and routes incoming inquiries to the right place on the team end. It has two primary components. The first is an AI-powered support agent that reads each inquiry, classifies it, handles routine questions directly, and (most importantly) recognizes when it is uncertain or out of its depth and hands the conversation off to a human rather than guessing. The second is a dashboard that gives the team two kinds of insight: operational visibility into what customers are asking about and how often, and a measure of how well the agent's decisions hold up against a labeled benchmark set.

Two ideas drive the design. The first concerns where language models fall short. Large language models generate fluent language well, but they have historically been unreliable at the decisions that actually move a task-oriented conversation forward: when to ask a clarifying question, when enough has been said to act, and when to stop and hand the conversation to a person. Left implicit inside a single model's text generation, these timing decisions are exactly where task-oriented bots tend to break down. This system therefore separates the two concerns: the language model handles wording, while a small, dedicated component owns the strategy of whether to answer, ask, or escalate at each turn. Making that decision explicit, rather than hoping the language model gets it right, is what lets the agent escalate reliably instead of confidently guessing.

The second idea is accessibility under limited compute. Because the control logic sits in a small dedicated component running over a small language model, the whole system stays light enough to run on ordinary hardware. It is designed to be domain-agnostic and self-hostable, so a business of any size can use it: a local computer parts store with a modest website can configure it to its own products and run it on its own machine, instead of paying for a hosted enterprise service, and the same architecture is built to scale up for larger support teams. Keeping response times low and resource needs modest is what makes the tool usable by businesses a remotely hosted service would price out.

This document covers the system's scope, architecture, data model, interfaces, and the key design decisions behind it.

2 System Architecture

2.1 Overview

The system is built as two cooperating services plus a React frontend that runs in the user's browser. The application service handles everything users touch and all operational data. The inference service handles the model and the answer-or-escalate decision. The frontend talks to the application service, the application service calls the inference service when it needs a classification, and each service owns its own data store. Every piece runs in its own container, and the whole stack can be started together on a single machine.

*Following Atlassian's guide to creating software design documents.

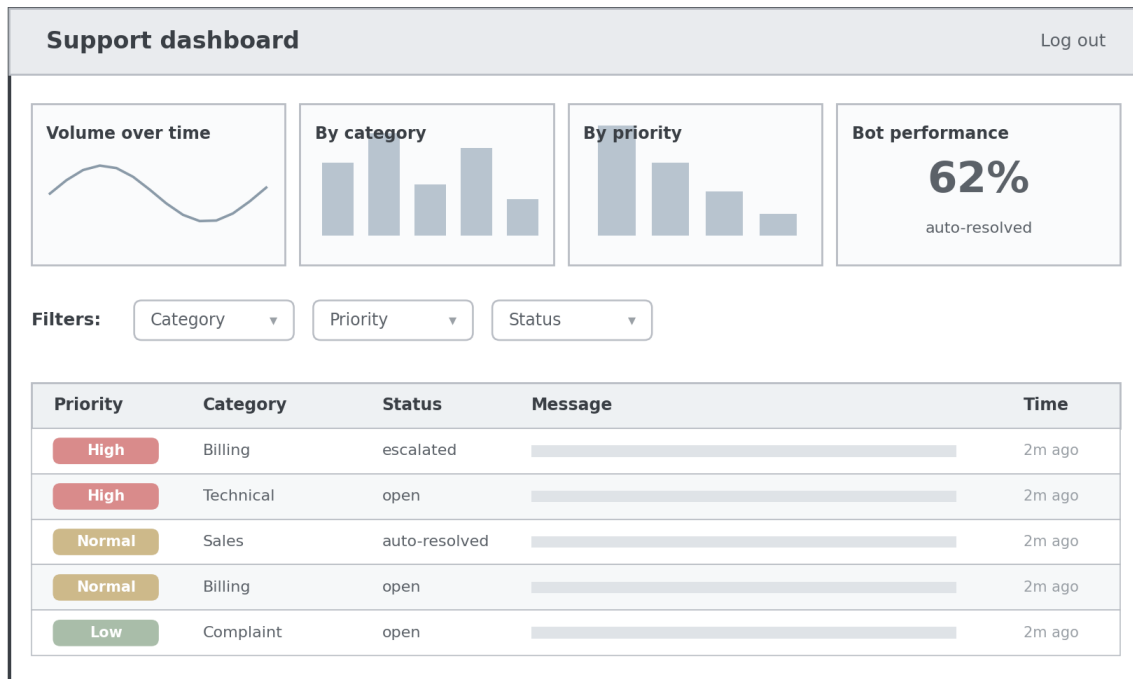


Figure 1: mock wireframe of the support team dashboard

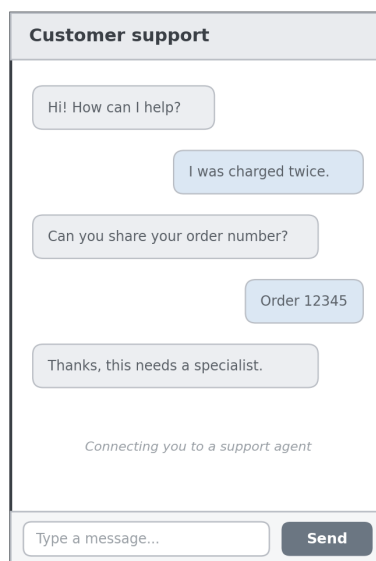


Figure 2: mock wireframe of the chat screen

3 Major components

- **React frontend.** The only part users see. It serves two faces from one application: the customer’s chat window and the support team’s dashboard.
- **Node.js application backend.** The coordinator. It receives inquiries, applies the routing rules, manages the queues, serves the dashboard its data, and calls the inference service for classifications. It runs on Node because the workload is mostly waiting on the database and the inference service, which Node handles well, and because it shares a language with the React frontend.
- **PostgreSQL. The operational record.** It stores the structured, related data that must stay consistent: tickets, categories, queues, and routing decisions, along with the figures behind the dashboard’s charts.
- **Spring Boot inference service. The decision maker.** Given a message, it returns a category, a priority, a confidence score, and the choice to answer, ask, or escalate. It houses the small language model and the lightweight control component that owns that strategy decision. It runs as its own service so the compute-heavy work can be scaled and restarted independently of the web tier.
- **MongoDB. The log store.** It records every prediction the inference service makes: the incoming text, the model’s output, and its confidence. These records are semi-structured and their shape changes as the model evolves, which suits a flexible document store better than a rigid table.
- **The model. The small language model** that generates wording, run either inside the inference service through a Java model runtime, or as a separate Python process the service calls.

3.1 Architectural style and patterns

The system uses a client-server structure: the React client communicates with the Node server over HTTP. Above that, it follows a service-oriented split, separating the application from the inference work into two independently deployable services that talk over a REST API. This is the microservices pattern in miniature, the point being that the compute-heavy inference can scale on its own without dragging the lightweight web tier along.

A third pattern operates inside the inference service: a separation between language generation and dialogue control. The language model handles wording, while a small dedicated component owns the decision of whether to answer, ask, or escalate. Making that decision explicit, rather than leaving it implicit in the model’s output, is what allows the agent to escalate reliably instead of guessing.

3.2 Key design decisions and trade-offs

Two services instead of one application. A single combined application would be simpler to build and deploy. The split is chosen so the expensive inference work can be scaled and updated independently of the web tier, and so the model runtime stays isolated from the web stack. The trade-off is added operational complexity, two services and a network call between them, in exchange for independent scaling. At small scale a single application would be sufficient, so this is a deliberate design-for-scale choice rather than a present necessity.

Two data stores, PostgreSQL and MongoDB. The operational ticket data is structured, related, and consistency-sensitive, which fits a relational database. The prediction logs are high-volume and shape-shifting, which fits a document store. The trade-off is running two data stores rather than one. The system could run on PostgreSQL alone, using its JSONB column type for the flexible logs; the two-store design is chosen to match each data type to the store best suited to it.

Spring Boot and Java for the inference service. The inference work could have been written in Python with FastAPI, which would pair naturally with a Python-native model. Java and Spring Boot are chosen because the compute-heavy workload suits Java better than the JavaScript backend would, and because using a different language for a separate service is a standard polyglot-microservices pattern. The trade-off is a third language in the stack, accepted in order to demonstrate an enterprise-standard backend skill.

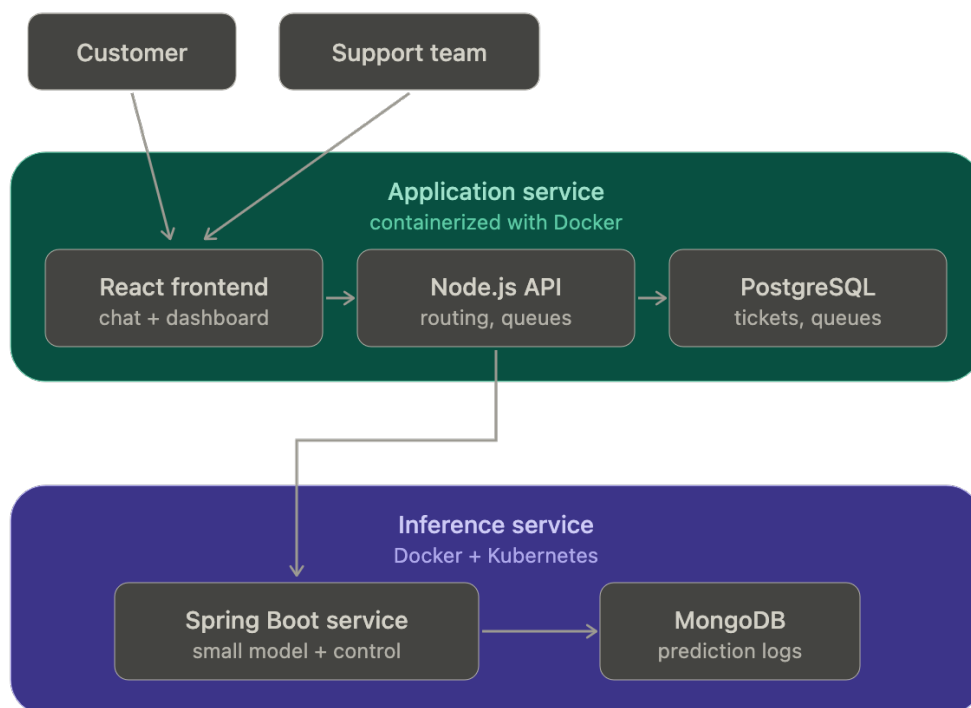


Figure 3: the architecture diagram, showing the two services and how a request flows through them

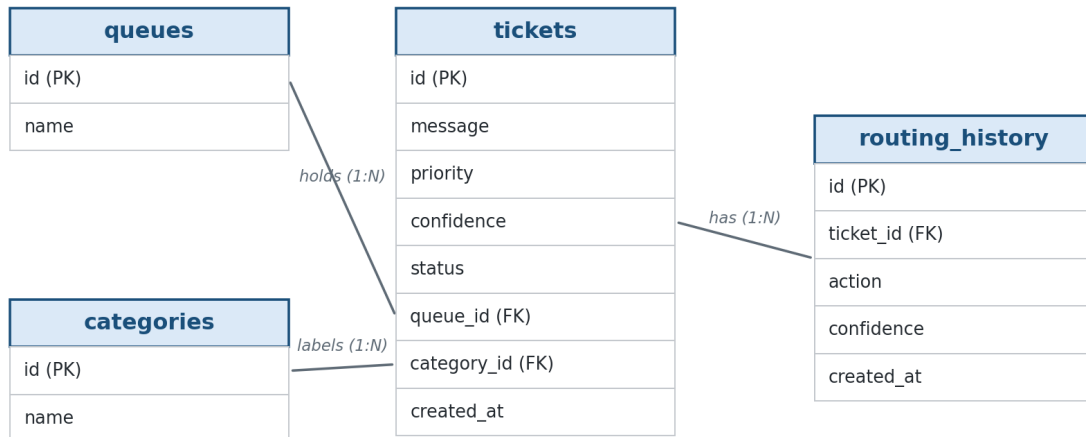


Figure 4: the entity-relationship diagram, showing the tables and how they link.

Containerization and orchestration. Every service runs in its own Docker container, which keeps each environment clean and lets the whole polyglot stack start with one command. Kubernetes manifests describe how the services would deploy and scale in production, demonstrated on a local cluster rather than a rented one. Kubernetes is more than a single-machine demo strictly requires; it is included to show the production deployment story, and the system runs equally well under Docker Compose on one machine for development.

Deployment note For development and demonstration, the entire stack runs locally: each service in its own container, started together with Docker Compose, with the model running on local hardware. The architecture is designed to scale out, but it has been validated at small scale, not under production load.

4 Database structure and table layouts

Operational data lives in PostgreSQL across four related tables. Prediction logs live in a single MongoDB collection.

The PostgreSQL tables:

- **tickets.** One row per incoming inquiry. Columns: id (primary key), message (the inquiry text), priority, confidence, status (open, auto-resolved, escalated, or closed), queue_id (foreign key to queues), category_id (foreign key to categories), and created_at. This is the central table; most of the system reads from and writes to it.
- **queues.** One row per destination a ticket can be routed to (billing, technical, complaints, and so on). Columns: id (primary key) and name.
- **categories.** One row per label a ticket can be given (billing, technical, sales, complaint). Columns: id (primary key) and name.
- **routing_history.** One row each time a ticket is routed. Columns: id (primary key), ticket_id (foreign key to tickets), action (answer, ask, or escalate), confidence, and created_at. Keeping a row per routing event preserves the history of how a ticket moved rather than overwriting it.

The relationships, shown in the diagram: one queue holds many tickets, one category labels many tickets, and one ticket has many routing_history rows. Those links are carried by the foreign keys: a ticket stores the id of its queue and its category, and a routing_history row stores the id of its ticket.

The MongoDB collection: `prediction_logs`.

One document per model prediction. Each document records the incoming text, the model version, the predicted category, priority, confidence, and the decision the model made, along with any extra fields a given model version produces. The collection is kept schema-flexible so that newer model versions can record additional fields without changing a fixed table.

Data flow:

A single inquiry moves through the system along the path shown in the architecture diagram. A customer message arrives at the Node backend. Node sends the text to the inference service, which classifies it and returns a category, priority, confidence, and a decision, while writing a full record of that prediction to the `prediction_logs` collection in MongoDB. Node then writes a new row to the `tickets` table in PostgreSQL with the returned category, priority, and confidence, assigns it to a queue, and records the routing event in `routing_history`. The dashboard reads back from PostgreSQL, using aggregate queries to produce the operational charts (volume by category, counts by priority, and so on), and from the MongoDB logs to produce the model-quality figures. So PostgreSQL is written on every new ticket and routing event and read heavily by the dashboard, while MongoDB is appended to on every prediction and read for performance reporting. Data validation and integrity rules Validation happens at two layers. The Node backend validates every incoming request before writing, rejecting malformed or incomplete inquiries so that bad data never reaches the database. PostgreSQL then enforces integrity at the storage layer:

Required fields are marked not null, so a ticket cannot be saved without its essential values. Allowed values are constrained, so priority and status can only hold values from their defined sets. Foreign keys are enforced, so a ticket cannot point at a queue or category that does not exist, and a `routing_history` row cannot reference a missing ticket. Primary keys guarantee every row has a unique identifier.

The MongoDB `prediction_logs` collection is treated as append-only: records are written once and never edited, which suits its role as a log of what the model did. How data is stored and retrieved The Node backend is the only component that reads and writes the PostgreSQL tables. It writes when a ticket is created or routed, and reads when the dashboard requests data, using aggregate SQL queries to compute the counts and trends the charts display. The inference service is the only component that writes to MongoDB, appending one document per prediction; those logs are read when the dashboard computes model-performance metrics. Keeping each store owned by a single service avoids two services writing the same data and keeps responsibilities clear.

5 Interface design

The system has two interfaces: the web app API, which the React frontend calls on the Node backend, and the inference API, which the Node backend calls on the Spring Boot service. Both use HTTP in the REST style, and both pass data as JSON.

5.1 API specifications and protocols

The web app API (frontend to Node) provides:

- **POST /messages:** submit a new customer message. The backend classifies it, decides whether to answer or escalate, stores the ticket, and returns the reply.
- **GET /tickets:** list tickets for the dashboard, with optional filters by category, priority, or status.
- **GET /tickets/{id}:** fetch one ticket and its routing history.
- **GET /dashboard/stats:** return the aggregated figures behind the charts (volume over time, counts by category and priority, and the bot's performance numbers).

The inference API (Node to Spring Boot) provides:

- **POST /classify:** submit a message and receive a category, priority, confidence, and the decision to answer, ask, or escalate.

The convention throughout: GET requests fetch data, POST requests submit or create it, and every endpoint does one job.

5.2 Message formats and data structures

All requests and responses are JSON. For example, a call to POST /classify sends:

```
{ "text": "I was charged twice for my order" }
```

and receives:

```
{ "category": "billing", "priority": "high",  
  "confidence": 0.91, "decision": "escalate" }
```

A call to POST /messages sends the customer's text and receives the bot's reply along with the id and status of the ticket it created. GET /dashboard/stats returns an object of named figures (counts and time series) that the dashboard renders into charts.

5.3 Error and exception handling

Every response carries a status marking it as a success or a specific kind of failure, following standard HTTP codes (a success code, a client-error code for bad input, a server-error code for an internal failure). Two cases matter most:

- **Bad input:** if a request is malformed or missing required fields, the backend rejects it with a client-error response and a short message, so invalid data never reaches the database.
- **Inference service unavailable:** if the classify call fails or times out, the backend does not drop the customer's message. It still stores the ticket, marks it as needing manual classification, and routes it to a human. A model outage degrades the system to manual handling rather than losing work.

5.4 Security and authentication

- The team-facing endpoints (dashboard data and ticket lists) require the support team to log in, so only authenticated staff can view or act on tickets. Requests carry a token that the backend checks on each call.
- The inference service is internal. It is reachable only by the Node backend, not exposed to the public internet, so no outside party can call the model directly.
- Secrets such as database credentials live on the server and are never sent to the browser, since the frontend is public and untrusted.

For v1, authentication is kept simple and applied to the team-facing routes. Full production hardening (rate limiting, fine-grained roles, audit logging) is noted as future work rather than built now.

6 Component design

This section looks at each major component on its own: its job, what goes in and out, the logic inside it, and what it depends on. The components are the React frontend, the Node backend, and the Spring Boot inference service. The two data stores are covered in the data design section and appear here only as dependencies.

6.1 React frontend

- **Purpose:** present the system to people. It serves the customer chat window and the support team dashboard.

- **Inputs and outputs:** it takes user actions (a typed message, a dashboard filter) and the data the backend returns, and renders the chat and the charts. It sends requests to the Node backend and displays the responses.
- **Processing logic:** display logic only. It holds no business rules; it shows whatever the backend gives it and forwards user actions.
- **Dependencies:** the Node backend, through the web app API.

6.2 Node backend

- **Purpose:** coordinate the system. It receives customer messages, gets them classified, routes them, manages the queues, and serves the dashboard its data.
- **Inputs and outputs:** in, a customer message or a dashboard request from the frontend; out, the bot's reply, a stored and routed ticket, or the aggregated figures for the charts.
- **Processing logic:** after the inference service returns a classification, the backend applies the routing rules. It maps the category to a queue, sets the priority, and uses the returned decision and confidence to choose whether to auto-resolve, route to a human, or open a bug. For the dashboard it runs aggregate queries over the ticket data to produce counts and trends.
- **Dependencies:** the inference service (for classification), PostgreSQL (for ticket and queue data), and, when a confirmed bug is routed, the external issue tracker.

6.3 Spring Boot inference service

- **Purpose:** turn a message into a decision. Given the text, it returns a category, a priority, a confidence score, and the choice to answer, ask, or escalate.
- **Inputs and outputs:** in, the message text (and the conversation so far); out, the classification and decision, returned to the Node backend and also written to the prediction log.
- **Processing logic:** two steps. First the small language model reads the message and produces a category and a confidence. Then a separate control component decides the action: if confidence is high and the issue is routine, answer or route directly; if confidence is low or the issue is sensitive, ask a clarifying question or escalate to a human rather than guess. Keeping this decision in its own component, instead of leaving it to the language model, is what makes the escalation behavior deliberate and tunable.
- **Dependencies:** the language model it runs, and MongoDB, where it logs each prediction.

7 User interface design

The system has two user-facing screens: the customer chat window and the support team dashboard. This section describes the layout of each, the steps a person takes to use them, and the accessibility requirements they must meet.

7.1 Key screens

Customer chat. A single, simple chat window. Most of the screen is the running conversation, showing the customer's messages and the bot's replies. A text box and a send button sit at the bottom. When the bot hands the conversation to a person, a short notice appears in the thread so the customer knows a human is taking over. Nothing else competes for attention; the screen does one thing.

Support team dashboard. The team's main screen. Across the top is a summary strip of charts: ticket volume over time, counts by category and priority, and the bot's performance figures. Below it is the ticket list, sorted by priority and filterable by category, priority, and status. Selecting a ticket opens its detail: the

original message, the assigned category and priority, the model's confidence, and the routing history. The queues are reachable as filters on this same list.

7.2 User workflows and interactions

Customer. The customer opens the chat and types a question. The bot replies, and may ask a clarifying question if it is unsure. If it can resolve the issue, it answers directly. If it cannot, or the matter is sensitive, it tells the customer it is connecting them to a person, and the conversation becomes a ticket placed in the right queue.

Support agent. The agent logs in and lands on the dashboard. They scan the summary charts for anything unusual, then open a queue or filter the ticket list. They select a ticket to see its full detail and history, and take action: reply, reassign it to another queue, or close it.

7.3 Accessibility considerations

- Everything is operable by keyboard alone, not only by mouse.
- Color is never the only signal. Priority and status carry a text label as well as a color, so they stay clear to color-blind users.
- Inputs, buttons, and charts carry proper labels and text alternatives, so a screen reader can announce them and the charts have a readable summary.
- Text uses legible sizes and sufficient contrast, and the layout adapts to smaller screens.

8 Assumptions and dependencies

8.1 Technical assumptions

- The system is developed and run on a single machine, with each service in its own Docker container and the full stack started together with Docker Compose.
- A small language model is used, one that runs on modest hardware (a single consumer GPU, or a CPU with enough memory), in line with the accessibility goal.
- Inputs are text-based support inquiries. For v1, English only.
- A deploying business can run the stack on its own hardware or on a basic cloud or free tier.

8.2 External dependencies

- Runtimes and frameworks: React, Node.js with Express, Spring Boot, PostgreSQL, MongoDB, Docker, and Kubernetes.
- The language model, run locally (for example through a local model runner) or, optionally, called through a hosted model API.
- Supporting libraries, such as a charting library for the dashboard.
- If the optional bug-routing feature is enabled, the API of an external issue tracker (for example GitHub).

8.3 Constraints

- The model is the heaviest component. Running it needs a modest GPU or sufficient CPU and memory, and most free hosting tiers cannot run it, so the model runs on local or self-provisioned hardware.
- v1 targets single-machine deployment. Horizontal scaling is designed for but not validated under production load.

- v1 handles text only. Voice and other channels are out of scope.

8.4 Regulatory and compliance considerations

- The system stores customer messages, which are personal data. A real deployment would need to handle that data in line with applicable rules (for example GDPR or CCPA). For v1 as a portfolio project, this is noted as a real-deployment requirement rather than implemented.
- Because the system is self-hostable, a business can keep customer data on its own hardware, which supports data-residency and privacy needs.

9 Glossary of terms

API The set of requests a service agrees to answer; its menu.

Endpoint One named address in an API that does a single job.

HTTP The standard way programs send requests and responses over the web.

REST A common style for organizing a web API around addresses and standard HTTP methods.

CRUD The four basic data operations: create, read, update, delete.

JSON A plain-text format of labeled fields, used to send data between services.

Frontend (client) The part of the app that runs in the user’s browser. Here, React.

Backend (server) A long-running program that listens for requests and answers them. Here, Node and Spring Boot.

Service An independently running program with one responsibility, such as the web app or the inference service.

Microservices Building a system as several small services that talk to each other over a network.

Container (Docker) A packaged unit holding a program together with its entire environment, so it runs the same anywhere.

Kubernetes A tool that runs, scales, and restarts many containers across one or more machines.

Relational database (SQL) A database of tables with fixed columns, where rows are linked by keys. Here, PostgreSQL.

NoSQL (document database) A database of flexible documents with no fixed schema. Here, MongoDB.

Primary key (PK) The unique id that names a single row in a table.

Foreign key (FK) A field that stores another row’s id, linking one table to another.

Language model (LLM) A model that reads and generates text. Here, a small one classifies messages.

Inference Running the model to produce a prediction.

Escalation (handoff) Passing a conversation from the bot to a human.

Wireframe A low-fidelity sketch of a screen’s layout.

CCPA The California Consumer Privacy Act, a California law giving residents rights over the personal data that businesses hold about them.

GDPR The General Data Protection Regulation, a European Union law governing how organizations collect, store, and use people’s personal data, and protecting their privacy.